

# WEEK 4 MODULE

Date : .....

## PROBLEM STATEMENT 1 :-

Given a positive integer  $N$ , determine the total number of digits present in the number. Write a program that counts & prints the total no. of digits.

Input Format :- A single integer  $N$ .

Output Format :- Print a single integer representing the total no. of digits in  $N$ .

Constraints :-  $1 \leq N \leq 10^{18}$

Logic :-

- Read the integer ' $N$ ' & initialize a variable  $count = 0$ . Repeat while  $N > 0$
- Remove the last digit  $CN = N/10$
- Increase count by 1
- When  $N$  becomes 0, count will be the total no. of digits. Then print 'count'.

Dry Run :- Input :- 12345

| Step     | Current N | NEW N | Count |
|----------|-----------|-------|-------|
| Start(0) | 12345     | 12345 | 0     |
| 1        | 12345     | 1234  | 1     |
| 2        | 1234      | 123   | 2     |
| 3        | 123       | 12    | 3     |
| 4        | 12        | 1     | 4     |
| 5        | 1         | 0     | 5     |

Output :- 5

Date : .....

Code :- #include <stdio.h>  
int main()

```
{
    long long n;
    scanf ("%lld", &n);
    int count = 0;
    if (n == 0) count = 1;
    while (n > 0)
    {
        count ++;
        n /= 10;
    }
    printf ("%d", count);
    return 0;
}
```

main.c



Share

Run

Output

```
1 #include <stdio.h>
2
3 int main() {
4     long long n;
5     scanf("%lld", &n);
6
7     int sum = 0;
8     while (n > 0) {
9         sum += n % 10;
10        n /= 10;
11    }
12
13    printf("%d\n", sum);
14    return 0;
15 }
```

12345

15

=== Code Execution Successful ===

Date : .....

## PROBLEM STATEMENT

Given a positive integer  $N$ , reverse it & print the reversed no.

WAP that reverses the given no.

Input format :- A single integer  $N$ .

Output format :- Print the reversed no.

Constraint :-  $1 \leq N \leq 10^9$

Logic :-

- Read the no.  $N$  & initialize  $reverse = 0$

- Repeat while  $N > 0$ :

- Get the last digit :-  $digit = N \% 10$

- Update reverse :-  $reverse = reverse * 10 + digit$

- Remove the last digit  $N = N / 10$

- Then print reverse

Dry Run :- Input :- 12345

| Step     | Digit | Reverse | N     |
|----------|-------|---------|-------|
| Start(0) | -     | 0       | 12345 |
| 1        | 5     | 5       | 1234  |
| 2        | 4     | 54      | 123   |
| 3        | 3     | 543     | 12    |
| 4        | 2     | 5432    | 1     |
| 5        | 1     | 54321   | 0     |

Output :- 54321

Date : .....

Code :- #include <stdio.h>

int main ()

{

long long n;

scanf ("%lld", &n);

long long rev = 0;

while (n > 0)

{

rev = rev \* 10 + n % 10;

n /= 10;

}

printf ("%lld\n", rev);

return 0;

}

main.c



Share

Run

Output

```
1 #include <stdio.h>
2
3 int main() {
4     long long n;
5     scanf("%lld", &n);
6
7     long long rev = 0;
8     while (n > 0) {
9         rev = rev * 10 + n % 10;
10        n /= 10;
11    }
12
13    printf("%lld\n", rev);
14    return 0;
15 }
```

12345

54321

=== Code Execution Successful ===

Date : .....

### PROBLEM STATEMENT 3

Given a positive integer  $N$ , calc. the sum of all its digits

WAP that computes & prints the sum of the digits.

Input - A single integer  $N$

Output - Print a single integer representing the sum of all digits

Constraints:  $1 \leq N \leq 10^{18}$

- Logic :-
- Input the no.  $N$  & initialize  $sum = 0$
  - ~~Input~~ Repeat while  $N > 0$ :
  - Get the last digit :  $digit = N \% 10$
  - Add it to sum :  $sum = sum + digit$
  - Remove the last digit  $N = N / 10$
  - Print sum

Dry Run :- Input: 12345

| Step      | Digit | Sum | N     |
|-----------|-------|-----|-------|
| Start (0) | -     | 0   | 12345 |
| 1         | 5     | 5   | 1234  |
| 2         | 4     | 9   | 123   |
| 3         | 3     | 12  | 12    |
| 4         | 2     | 14  | 1     |
| 5         | 1     | 15  | 0     |

Output :- 15

Date : .....

```
Code :- #include <stdio.h>
int main()
{
    long long n;
    scanf("%lld", &n);
    int sum = 0;
    while (n > 0)
    {
        sum += n % 10;
        n /= 10;
    }
    printf("%d\n", sum);
    return 0;
}
```

main.c



Share

Run

Output

```
1  #include <stdio.h>
2
3  int main() {
4      long long n;
5      scanf("%lld", &n);
6
7      int sum = 0;
8      while (n > 0) {
9          sum += n % 10;
10         n /= 10;
11     }
12
13     printf("%d\n", sum);
14     return 0;
15 }
```

12345

15

=== Code Execution Successful ===

## Date :.....

Date : .....

- Initialize two variables: even = 0, odd = 0
- Repeat while  $N > 0$ 
  - Find the last digit:  $digit = N \% 10$
  - If  $digit \times 2 == 0$ , increment even, else increment odd.
- Remove the last digit  $N = N / 11$
- Print even odd.

| Step     | Digit | Even | Odd | N      |
|----------|-------|------|-----|--------|
| Send (0) | -     | 0    | 0   | 123456 |
| 1        | 6     | 1    | 0   | 12345  |
| 2        | 5     | 1    | 1   | 1234   |
| 3        | 4     | 2    | 1   | 123    |
| 4        | 3     | 2    | 2   | 12     |
| 5        | 2     | 3    | 2   | 1      |
| 6        | 1     | 3    | 3   | 0      |

**Case 1**

```

1 long long n;
2 scanf ("%lld", &n);
3 int even = 0, odd = 0;
4 while (n > 0)
5 {
6     int digit = n % 10;
7     if (digit % 2 == 0)
8     {
9         even++;
10    }
11    else
12    {
13        odd++;
14    }
15    n /= 10;
16 }
17 printf ("%d %d\n", even, odd);
18 return 0;
19 }

```

*Spiral*



main.c



Share

Run

Output

```
1 #include <stdio.h>
2
3 int main() {
4     long long n;
5     scanf("%lld", &n);
6
7     int even = 0, odd = 0;
8     while (n > 0) {
9         int digit = n % 10;
10        if (digit % 2 == 0) {
11            even++;
12        } else {
13            odd++;
14        }
15        n /= 10;
16    }
17
18    printf("%d %d\n", even, odd);
19    return 0;
20 }
```

123456

3 3

=== Code Execution Successful ===



Date : .....

## PROBLEM STATEMENT 5

Given a statement integer  $N$  find & print the largest digit present in the no. WAP that determines the max. digit

Input :- A single integer  $N$ .

Output :- Print the largest digit present in the given no.

Constraints :-  $1 \leq N \leq 10^8$ .

Logic :- Read the integer  $N$ , and initialize  $\text{max\_digit} = 0$

- Repeat while  $N > 0$  :
  - Find the last digit :  $\text{digit} = N \% 10$
  - If  $\text{digit} > \text{max\_digit}$ , update  $\text{max\_digit}$
  - Remove the last digit :  $N = N / 10$
  - Print  $\text{max\_digit}$

Dry Run :- Input :- 472953

| Step  | Digit | Max | N      |
|-------|-------|-----|--------|
| Start | -     | 0   | 472953 |
| 1     | 3     | 3   | 47295  |
| 2     | 5     | 5   | 4729   |
| 3     | 9     | 9   | 472    |
| 4     | 2     | 9   | 47     |
| 5     | 7     | 9   | 4      |
| 6     | 4     | 9   | 0      |

Output :- 9

Date : .....

Code :- #include <stdio.h>  
int main()

```

{
    long long n;
    scanf("%lld", &n);
    int max_digit = 0;
    while (n > 0)
    {
        int digit = n % 10;
        if (digit > max_digit)
        {
            max_digit = digit;
        }
        n /= 10;
    }
    printf("%d\n", max_digit);
    return 0;
}

```

main.c



Share

Run

Output

```
1  #include <stdio.h>
2
3  int main() {
4      long long n;
5      scanf("%lld", &n);
6
7      int max_digit = 0;
8      while (n > 0) {
9          int digit = n % 10;
10         if (digit > max_digit) {
11             max_digit = digit;
12         }
13         n /= 10;
14     }
15
16     printf("%d\n", max_digit);
17     return 0;
18 }
```

472953

9

=== Code Execution Successful ===

Date : .....

## PROBLEM STATEMENT - 6

Given two positive integer  $L$  &  $R$  count how many prime no. are present in the inclusive range  $[L, R]$ .

WAP that counts and prints the total no. of prime no.s in the given range.

Input :- Two integers  $L$  &  $R$  separated by a space.

Output :- Print a single integer representing the total no. of prime.

Constraint :-  $1 \leq L \leq R \leq 10^5$

Logic :-

- Read  $L$  &  $R$  & initialize count = 0
- for each number from  $L$  to  $R$ :

- check whether it is prime
- If prime, increment count
- prime count

Dry Run :- Input :- 1 10

| Number | Prime | count |
|--------|-------|-------|
| 1      | No    | 0     |
| 2      | Yes   | 1     |
| 3      | Yes   | 2     |
| 4      | No    | 2     |
| 5      | Yes   | 3     |
| 6      | No    | 3     |
| 7      | Yes   | 4     |
| 8      | No    | 4     |
| 9      | No    | 4     |
| 10     | No    | 4     |

Date : .....

Count :- #include <zetol.h>

```
int is_prime(int num)
{
    if (num <= 1)
        return 0;
    for (int i = 2; i <= num; i++)
    {
        if (num % i == 0)
            return 0;
    }
    return 1;
}
```

```
int main()
{
    int L, R;
    scanf("%d %d", &L, &R);
    int count = 0;
    for (int i = L; i <= R; i++)
    {
        if (is_prime(i))
            count++;
    }
    printf("%d", count);
    return 0;
}
```

main.c



Share

Run

Output

```
1  #include <stdio.h>
2
3  int is_prime(int num) {
4      if (num <= 1) return 0;
5      for (int i = 2; i * i <= num; i++) {
6          if (num % i == 0) return 0;
7      }
8      return 1;
9  }
10
11 int main() {
12     int l, r;
13     scanf("%d %d", &l, &r);
14
15     int count = 0;
16     for (int i = l; i <= r; i++) {
17         if (is_prime(i)) {
18             count++;
19         }
20     }
21
22     printf("%d\n", count);
23     return 0;
24 }
```

11 20

4

=== Code Execution Successful ===

Date : .....

## PROBLEM STATEMENT :-

Given a positive integer  $N$ , find the sum of all its proper divisors.

WAP that calc. & prints the sum of proper divisors of  $N$ .

Input :- A single integer  $N$

Output :- Print a single integer representing the sum of all proper divisors.

Constraint :-  $1 < N < 10^6$

Logic :-

- Read  $N$  & initialize sum = 0
- Loop from 1 to  $N-1$
- If  $N \% i == 0$ , add it to sum
- Print sum

Dry Run Input :- 12

| Number | Divisor? | Sum |
|--------|----------|-----|
| 1      | Yes      | 1   |
| 2      | Yes      | 3   |
| 3      | Yes      | 6   |
| 4      | Yes      | 10  |
| 5      | No       | 10  |
| 6      | Yes      | 16  |
| 7-11   | No       | 16  |

Output :- 16

Date : .....

Code :-

```
#include <stdio.h>
int main()
{
    int n;
    if (n > 1)
    {
        sum = 1;
        for (int i = 2; i <= n; i++)
        {
            if (n % i == 0)
            {
                if (i != n/i)
                {
                    sum += n/i;
                }
            }
        }
        printf("%d\n", sum);
        return 0;
    }
}
```

main.c



Share

Run

Output

```
1 #include <stdio.h>
2
3 int main() {
4     int n;
5     scanf("%d", &n);
6
7     long long sum = 0;
8     if (n > 1) {
9         sum = 1;
10        for (int i = 2; i * i <= n; i++) {
11            if (n % i == 0) {
12                sum += i;
13                if (i != n / i) {
14                    sum += n / i;
15                }
16            }
17        }
18    }
19
20    printf("%lld\n", sum);
21    return 0;
22 }
```

12

16

=== Code Execution Successful ===

Date : .....

## PROBLEM STATEMENT - 8

Given a positive integer  $N$ , print all the factors of the no. in ascending order.

WAP that prints all the factors of  $N$ .

Input :- A single integer  $N$ .

Output :- Print all the factors of  $N$  in ascending order.

Constraints :-  $1 \leq N \leq 10^6$

- Logic :-
- Read the integer  $N$ .
  - Loop from 1 to  $N$ .
  - If  $N \% i == 0$ , print  $i$ .
  - Continue until the loop ends.

Dry Run :- Input :- 12

| Number | $12 \% i == 0$ | Output      |
|--------|----------------|-------------|
| 1      | Yes            | 1           |
| 2      | Yes            | 12          |
| 3      | Yes            | 12 3        |
| 4      | Yes            | 12 3 4      |
| 5      | No             | 12 3 4      |
| 6      | Yes            | 12 3 4 6    |
| 7-11   | No             | 12 3 4 6    |
| 12     | Yes            | 12 3 4 6 12 |

.....

Date : .....

Code :- #include <stdio.h>  
int main()

```

{
    int n;
    scanf ("%d", &n);
    for (int i=1; i<=n; i++)
    {
        if (n % i == 0)
        {
            printf ("%d ", i);
        }
    }
    printf ("\n");
    return 0;
}

```



main.c



Share

Run

Output

```
1  #include <stdio.h>
2
3  int main() {
4      int n;
5      scanf("%d", &n);
6
7      for (int i = 1; i <= n; i++) {
8          if (n % i == 0) {
9              printf("%d ", i);
10         }
11     }
12     printf("\n");
13
14     return 0;
15 }
```

12

1 2 3 4 6 12

=== Code Execution Successful ===

23-24/24

